

AI CUSTOMER SUPPORT AGENT

Amazon Help on Twitter

Intent classification • Retrieval-grounded replies • Human escalation

54.64%	Intent accuracy
0.45	Intent macro F1
69.42%	Escalation accuracy

Executive Summary

This project builds an AI-powered customer support agent for Amazon's Twitter support handle, Amazon Help. The system takes an incoming customer message and performs three tasks: classify the support intent, draft a reply grounded in real historical resolutions, and decide whether the case can be safely auto-handled or should be escalated to a human agent.

Amazon Help was selected because the dataset contains a high volume and broad variety of realistic support scenarios. The implementation focuses on a representative subset of the approximately 3M-tweet corpus so the pipeline remains reproducible and fast to iterate.

The headline result is deliberately mixed: the LLM classifier substantially beats both simple baselines, while important weaknesses remain. Common operational intents such as delivery delay, refunds, damaged items, and payment issues perform strongly; product/content requests remain a major failure area. Escalation also shows a meaningful precision–recall trade-off.

Core principle: optimize for evidence and reliability, not a flattering headline number.

1. Problem Framing & Data

The system is designed around a realistic support workflow rather than a generic chatbot. The incoming customer message is interpreted, connected to historical support behavior, and routed according to risk.

System responsibilities

- **Intent classification:** assign one support intent from a compact taxonomy derived from real Amazon Help conversations.
- **Reply drafting:** retrieve similar customer complaints and the actual Amazon replies associated with them, then use those examples as grounding context.
- **Escalation:** determine whether an AI response is appropriate or a human should take over, with an explicit reason.

Data preparation

The raw Customer Support on Twitter dataset was filtered to Amazon Help's replies and joined back to the original customer tweets using the response relationship, producing customer → Amazon conversation pairs.

An ASCII-ratio heuristic (>0.9 ASCII characters) was used as a fast English-language/noise filter. This is not true language detection: Latin-script French, German, Portuguese, Italian, and Spanish messages can pass through. That limitation later surfaced as a real source of error.

Evaluation and retrieval data

Component	Choice
Golden set	280 hand-labeled development examples; final submission should use an official 250-row subset.
Retrieval corpus	15,000 historical customer → Amazon reply pairs
Embedding	all-MiniLM-L6-v2
Similarity	Cosine similarity
LLM	Nemotron via NVIDIA NIM / OpenAI-compatible API

Intent taxonomy

The taxonomy started with ten categories and grew to thirteen during labeling as real examples exposed missing distinctions: **delivery_delay**, **order_not_received**, **refund_return**, **damaged_or_wrong_item**, **account_access**, **payment_issue**, **prime_subscription**, **product_content_request**, **customer_service_escalation**, **technical_issue**, **follow_up_acknowledgment**, **non_english**, **general_other**.

Escalation is modeled separately as a boolean attribute because a message can have an intent such as *refund_return* while independently requiring human review.

2. System Architecture & Method

End-to-end flow

Incoming message → Intent classifier → Historical retrieval → Grounded reply generation → Escalation policy → Final action.

13	Intent categories
15,000	Historical cases
Top-3	Grounding examples

A. Intent classification

A hosted LLM receives the customer message and the complete allowed label list. The prompt requires exactly one label and no explanation. Temperature is set to zero. A post-processing layer removes leaked thinking tags and whitespace artifacts and maps outputs back to the known label set.

B. Retrieval-grounded reply generation

Historical customer messages are embedded using **all-MiniLM-L6-v2**. For a new message, cosine similarity selects the top three semantically similar complaints and their real Amazon replies. These examples are placed in the generation prompt so the model can imitate observed support tone and resolution style rather than answering from an unconstrained knowledge prior.

The generation prompt asks for concise, empathetic, helpful replies and prohibits inventing order numbers or unavailable details. Manual spot checks covered delivery, refund, account access, product questions, and angry repeat-contact.

C. Layered escalation

Fixed rules immediately escalate high-risk intents such as **account_access**, **refund_return**, and **customer_service_escalation**, plus obvious repeated-frustration signals. Other cases are passed to the LLM, which returns a binary decision and a one-sentence reason.

Reliability considerations

- The API key is stored in Colab Secrets rather than hardcoded.
- Model output is validated before evaluation instead of assuming every response is parseable.
- Retrieval grounds generation in real historical replies, but insufficient evidence can still create unsupported factual claims; this remains a generation risk.
- The system is evaluated on hand-labeled examples rather than relying only on qualitative demos.

3. Results & Baselines

Headline evaluation

54.64%	LLM intent accuracy
0.45	Intent macro F1
69.42%	Escalation accuracy

Intent classification vs. baselines

Approach	Accuracy	Interpretation
Trivial baseline	28.21%	Always predicts delivery_delay
Keyword baseline	19.29%	Hand-written keyword rules with general_other fallback
Nemotron classifier	54.64%	Zero-shot LLM using the full taxonomy

The LLM improves by **26.43 percentage points** over the trivial baseline and **35.35 points** over keyword matching on the development evaluation. The keyword baseline performs worse than the trivial baseline because overlapping vocabulary causes false matches and many messages fall back to general_other.

Per-category performance

Category	F1	Readout
delivery_delay	0.85	Strong
refund_return	0.84	Strong
damaged_or_wrong_item	0.80	Strong
payment_issue	0.76	Strong
technical_issue	0.62	Mixed
prime_subscription	0.56	Mixed
account_access	0.35	Weak
customer_service_escalation	0.35	Weak
non_english	0.38	Weak
order_not_received	0.36	Weak
follow_up_acknowledgment	0.27	Weak
general_other	0.23	Weak
product_content_request	0.00	Failure

4. Failure Analysis & What the Headline Hides

Top 5 failure modes

- **Product/content requests — 0% F1.** The classifier never predicted this category in the evaluated examples. These messages have less consistent wording than delivery complaints and are absorbed into more common classes.
- **General/other as a dumping ground.** The catch-all class is overused when the model is uncertain, hiding several underlying problem types.
- **Escalation under-recall — 54%.** Escalate precision was 0.79, but recall was only 0.54, so a substantial fraction of human-review cases are missed.
- **Language leakage.** The ASCII-ratio filter allowed several European Latin-script messages through. Seven examples were subsequently adjudicated as non-English, exposing a preprocessing limitation.
- **Malformed model output.** Some responses leaked thinking markers or whitespace variants. Without cleanup, valid classes could be fragmented during scoring.

Escalation detail

0.79	Escalate precision
0.54	Escalate recall
0.64	Escalate F1

This trade-off matters operationally. A missed escalation can be more costly than an unnecessary human handoff. The next iteration should therefore tune toward higher escalation recall rather than optimizing raw accuracy alone.

What is misleading about 54.64%?

The single accuracy number is simultaneously too pessimistic for common operational intents and too optimistic as a description of the full system. Delivery delay, refunds, damaged items, and payment issues have F1 scores of 0.76–0.85, while product/content requests are at zero. Accuracy also gives every error equal weight even though the operational cost of a missed escalation can be much higher than a low-stakes routing error.

The number says nothing about generated-reply quality or whether escalation is appropriate from a human-support perspective. Those customer-facing outcomes require separate evaluation.

Evaluation caveat

The development run used 280 hand-labeled examples as a buffer. For final submission, the official golden evaluation should be fixed at 250 examples to match the assignment's stated 150–250 range; remaining examples should be treated as reserve.

5. Next-Week Plan & Submission

The prototype has a complete support-agent loop with measurable strengths and clear weaknesses. The next iteration should reduce high-impact errors rather than add unnecessary infrastructure.

Priority 1 — Fix product/content routing

Add targeted few-shot examples and review whether product/content is too broad. Separate product information, digital content, availability, and related questions from `general_other` and `technical_issue`.

Priority 2 — Increase escalation recall

Tune the rule layer and LLM prompt toward safer escalation. Re-measure precision and recall, treating missed true escalations as the higher-cost error.

Priority 3 — Complete reply-quality evaluation

Create a human-rated sample of generated replies and compare it with an automated LLM-as-judge score. Validate the judge against human agreement before reporting automated quality numbers.

Priority 4 — Replace the language heuristic

Replace the ASCII-ratio filter with a proper language detector and re-score the golden set. This separates genuine model confusion from preprocessing-induced errors.

Priority 5 — Strengthen the evaluation set

Add more examples for underrepresented categories so per-category F1 becomes more statistically reliable.

Recommended submission package

- **Report PDF:** methodology, evaluation, failure analysis, and next-week plan.
- **Notebook:** reproducible data preparation, retrieval, classification, generation, escalation, and evaluation.
- **Golden evaluation file:** final 250-row labeled set with predictions and escalation outputs.
- **README:** concise project overview, setup instructions, how to run, and results.

Decision Log

A separate professional decision log will document the key engineering choices: brand selection, filtering, taxonomy evolution, model selection, retrieval design, escalation policy, evaluation cleanup, baseline selection, and security choices.

Bottom line: the prototype demonstrates a complete support-agent loop and beats both simple baselines, while exposing meaningful gaps. The strongest next step is to fix the high-impact failures and prove the improvement with a cleaner evaluation.